Lab 3: Local search

Authors: Piotr Foltyniewicz 160295, Jakub Adamski 160291

Problem description

Each instance consists of 200 nodes. A row in the input file contains the node's planar coordinates and selection cost, in the form `x;y;cost`. Node indices are zero-based and correspond to row order. The task is to select exactly 100 distinct nodes (half of the available nodes), order them, and form a closed cycle that minimizes the sum of travel distances and selected-node costs.

Distances are Euclidean, rounded to the nearest integer and precomputed after reading the instance. All algorithms use this distance matrix. The evaluation sums all selected-node costs and all cycle edges, including last -> first. In the pseudocode, `m = 100`, `n = 200`, `c[v]` is node cost, and `d(a, b)` is the precomputed distance. A solution `S` is an ordered list of `m` distinct nodes; `U` denotes the set of nodes not in `S`.

The objective for an ordered selection `S` is:

```
F(S) = sum(c[S[i]]) + sum(d(S[i], S[(i + 1) mod length(S)]))
```

Pseudocode

All indices are zero-based. Local search starts from a given solution and applies moves until none improves the objective. Moves are evaluated only through deltas; the neighbor solution is never built and `F` is never recalculated from scratch. The eight methods combine: steepest or greedy search, node or edge exchange as the intra-route move, random or greedy starting solution.

Neighborhood and deltas

The neighborhood consists of inter-route moves and one kind of intra-route moves. A move is only a descriptor (type and positions).

- `INTER(p, v)`: replace the selected node at position `p` with an unselected node `v`.
- `SWAP_NODES(i, j)`, `i < j`: exchange the nodes at positions `i` and `j`.
- `SWAP_EDGES(i, j)`, `i < j`: remove edges `(S[i], S[i+1])` and `(S[j], S[j+1])` and reverse `S[i+1..j]`. Adjacent edges are skipped.

Positions are taken modulo `m`, so the closing edge is handled like the others.

```
DELTA_INTER(S, p, v):
    out = S[p]; prev = S[p - 1]; next = S[p + 1]
    return d(prev, v) + d(v, next) - d(prev, out) - d(out, next) + c[v] - c[out]

DELTA_SWAP_EDGES(S, i, j):
    a = S[i]; b = S[i + 1]; e = S[j]; f = S[j + 1]
    return d(a, e) + d(b, f) - d(a, b) - d(e, f)

DELTA_SWAP_NODES(S, i, j):
    E = distinct edge starts among {i - 1, i, j - 1, j}
```

```
    return sum over e in E of d'(e, e + 1) - d(e, e + 1)     // d' = edge length after the
swap
```

Steepest local search

```
STEEPEST(S):
    loop:
        best = NONE; bestDelta = 0
        for move in all moves:
            delta = DELTA(S, move)
            if delta < bestDelta: best = move; bestDelta = delta
        if best == NONE: return S
        APPLY(S, best)
```

Greedy local search

Descriptors of all moves of both kinds are put into one list, which is shuffled uniformly (`std::shuffle` , `mt19937` seeded by `random_device`), so the two kinds are mixed. Deltas are not precomputed: moves are evaluated lazily in the shuffled order and the first improving one is applied. After each applied move the list is generated and shuffled again.

```
GREEDY(S):
    loop:
        moves = all moves, shuffled
        for move in moves:
            if DELTA(S, move) < 0:
                APPLY(S, move); goto next iteration
        return S
```

Starting solutions

Random: the first `m` nodes of a random permutation of all `n` nodes. Greedy: the best construction heuristic of labs 1-2 for the instance, started from each of the 200 nodes: weighted-sum greedy cycle for TSPA and weighted-sum nearest neighbor for TSPB.

Each method was run 200 times per instance (200 random solutions, or each node as the construction start). Times include the construction of the starting solution.

Results

Each method was tested 200 times on TSPA and TSPB. Lower objective values are better. Results of labs 1 and 2 are copied from the previous reports. Methods of this lab are in bold. Tables are sorted by the mean objective; times are averages per run. For lab 3, "Starting solution" is the average time of generating the starting solution (the greedy construction heuristic, or a random permutation) and "Local search" is the remainder of the run (Avg Time minus Starting solution).

TSPA

Method	Runs	Minimum	Maximum	Mean	Avg Time (ms)	Starting solution	Local search

						(ms)	(ms)
Greedy Edges Greedy	**200**	**71955**	**74877**	**72986.99**	**1.761381**	**1.330380**	**0.431001**
Steepest Edges Greedy	**200**	**71955**	**74861**	**73003.80**	**1.647100**	**1.330380**	**0.316720**
Steepest Nodes Greedy	**200**	**71955**	**74861**	**73021.71**	**1.887184**	**1.330380**	**0.556804**
Greedy Nodes Greedy	**200**	**71955**	**74877**	**73023.20**	**1.871932**	**1.330380**	**0.541553**
Greedy Cycle heuristics with a weighted sum criterion	200	72011	75532	73353.51	5.075577	–	–
Greedy Cycle	200	72590	75281	73743.73	3.582711	–	–
Nearest Node Any	200	72328	77022	74306.53	2.884855	–	–
NN heuristics with a weighted sum criterion	200	72419	78504	75561.20	5.377398	–	–
Greedy Edges Random	**200**	**74032**	**79155**	**76132.63**	**19.880733**	**0.003564**	**19.877169**
Steepest Edges Random	**200**	**73752**	**80365**	**76182.19**	**9.860661**	**0.003564**	**9.857097**
Nearest Node End	200	84362	90075	87476.63	0.060209	–	–
Greedy Nodes Random	**200**	**82575**	**96252**	**88118.92**	**20.654955**	**0.003564**	**20.651390**

Steepest Nodes Random	**200**	**82617**	**98631**	**89713.89**	**20.137337**	**0.003564**	**20.133772**
Greedy Cycle 2-regret heuristics	200	112806	128089	120378.76	5.229371	–	–
NN heuristics 2-regret heuristics	200	110950	135771	121360.29	5.459556	–	–
Random	200	233146	288073	260752.93	0.007843	–	–

TSPB

Method	Runs	Minimum	Maximum	Mean	Avg Time (ms)	Starting solution (ms)	Local search (ms)
Steepest Edges Random	**200**	**50650**	**55430**	**52633.83**	**9.851301**	**0.003483**	**9.847818**
Greedy Edges Random	**200**	**50494**	**55929**	**53002.30**	**20.281064**	**0.003483**	**20.277582**
Steepest Edges Greedy	**200**	**49346**	**60354**	**53091.28**	**2.471579**	**1.354470**	**1.117109**
Greedy Edges Greedy	**200**	**49793**	**60266**	**53150.93**	**2.990122**	**1.354470**	**1.635652**
Greedy Nodes Greedy	**200**	**49913**	**61265**	**54223.62**	**2.619082**	**1.354470**	**1.264612**
Steepest Nodes Greedy	**200**	**49978**	**61265**	**54292.07**	**2.815516**	**1.354470**	**1.461046**
NN heuristics with a weighted sum criterion	200	50485	61406	55698.34	5.332107	–	–

Greedy Cycle heuristics with a weighted sum criterion	200	52731	63370	57866.33	5.140883	–	–
Nearest Node End	200	56007	63166	59397.33	0.086600	–	–
Nearest Node Any	200	52915	66164	59476.22	4.494223	–	–
Greedy Cycle	200	55655	63734	59650.81	3.416733	–	–
Greedy Nodes Random	**200**	**58992**	**70290**	**63882.50**	**20.400368**	**0.003483**	**20.396886**
Steepest Nodes Random	**200**	**58240**	**74951**	**65886.73**	**19.605030**	**0.003483**	**19.601547**
Greedy Cycle 2-regret heuristics	200	68150	77274	72484.84	5.307665	–	–
NN heuristics 2-regret heuristics	200	68761	79756	73649.84	5.374936	–	–
Random	200	179244	236859	206352.15	0.010232	–	–

Note: labs 1-2 were run on a different machine than lab 3, so their times are not directly comparable with the times of lab 3.

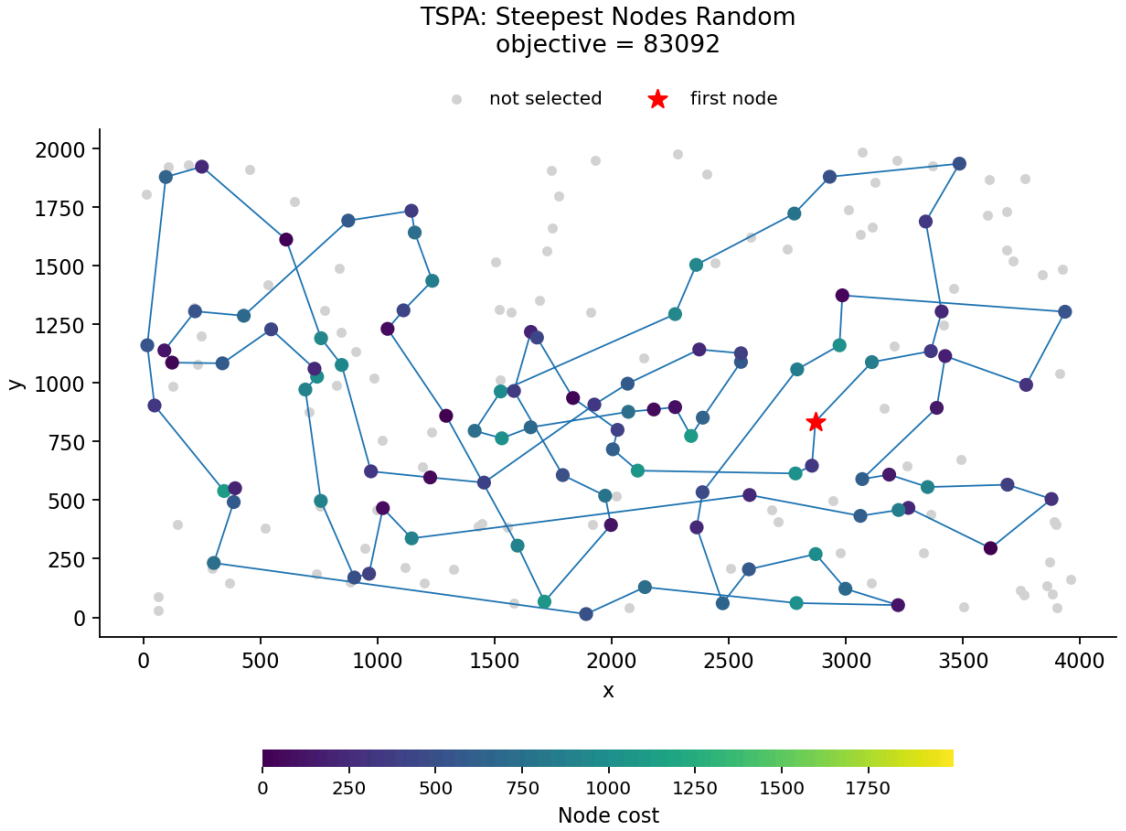
Best solutions and visualizations

Node color represents selection cost; the compact horizontal scale below each plot uses the same cost range for all methods on an instance. Gray nodes are not selected, blue segments form the closed tour, and the red star marks the first node in the displayed sequence. Lists below show ten node indices per row; read left to right, then top to bottom to follow the tour.

TSPA

Steepest Nodes Random

Objective: **83092**.

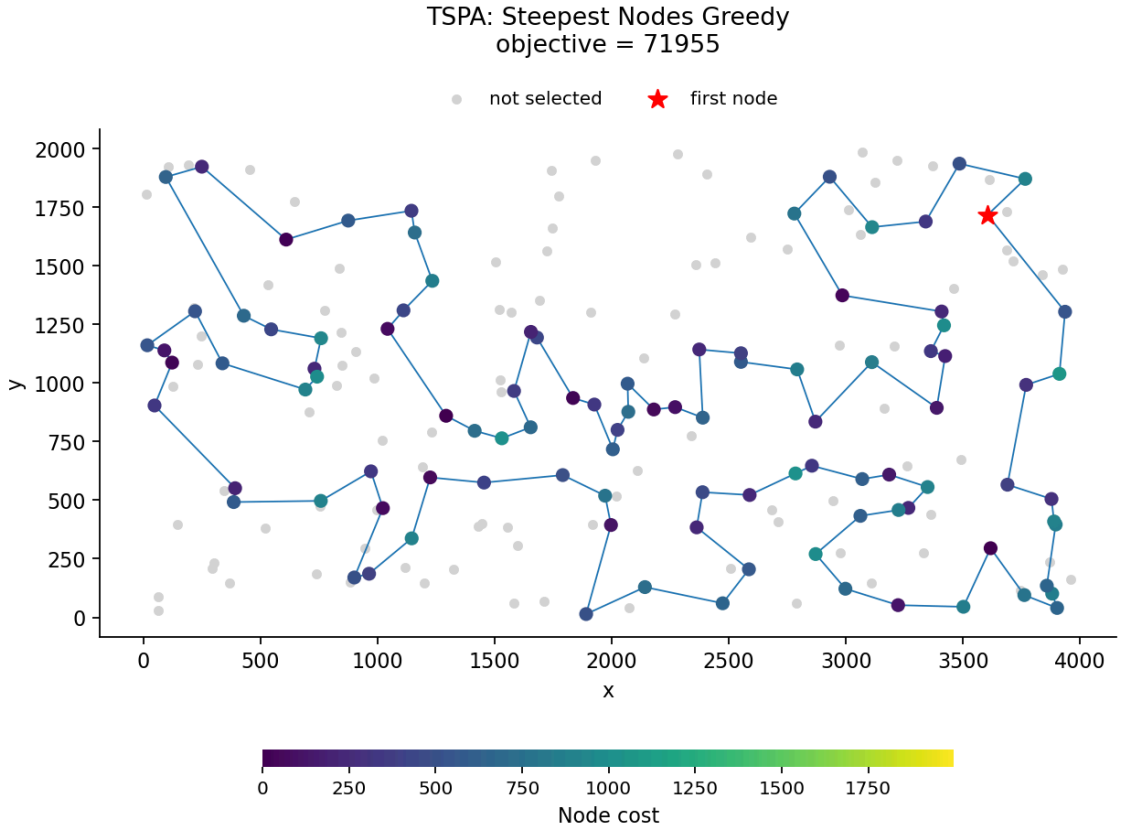


Solution (tour order):

```
174  85   3   1 178  29 136 186  62 143
156  28 102 185 179 177  36  99 131  54
  9  33  52 144  87 141  30  90 100  80
 48  31 116 168  95  97 127  69 114 181
111  57 182 138 158 192  58  84   8 142
164 147  47  50  93 150 106 113 130 159
107  89  42 151 134 189 187   2 171 161
160  83 123 152  49  63 149  60 108  91
157  71   4  76 132 180  98  25 135  73
 45  35  51  16  67  53 140  41  72 176
```

Steepest Nodes Greedy

Objective: **71955**.

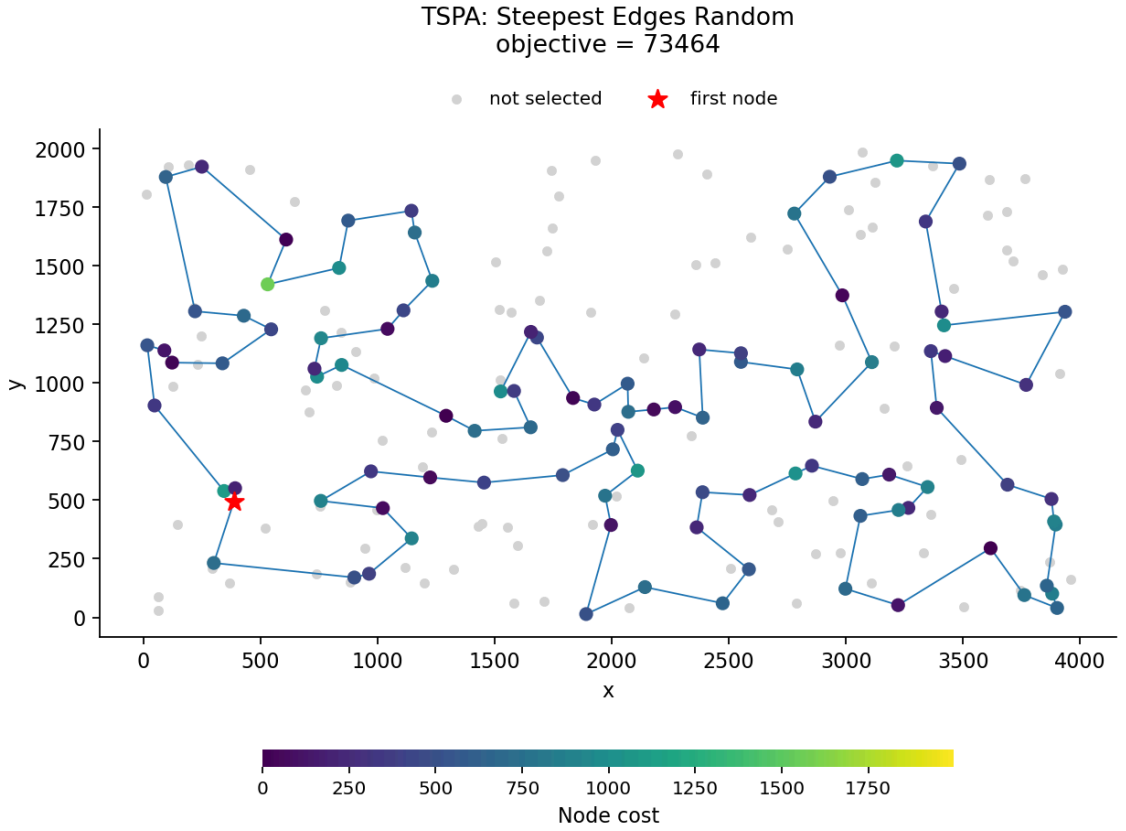


Solution (tour order):

```
101  55  29 178 184 136 186 164   1 169
  3  50  93  85 174   8  54   9  33 131
 36 177  52 179  53 140 144  67  16  51
 35 185 102  28 180 132  76   4  71 157
 91  80  48  31 108 152 100 123  83 160
 49  60 116 149  63 168  97 127 161  30
187 171   2 189 141  87  45  73 135 114
181 192 158  58  84 134  72 176 150 106
113  89  42 151 138 182  57  64 107  88
 78 193 153 170 146 159 130  47  37 147
```

Steepest Edges Random

Objective: **73464**.

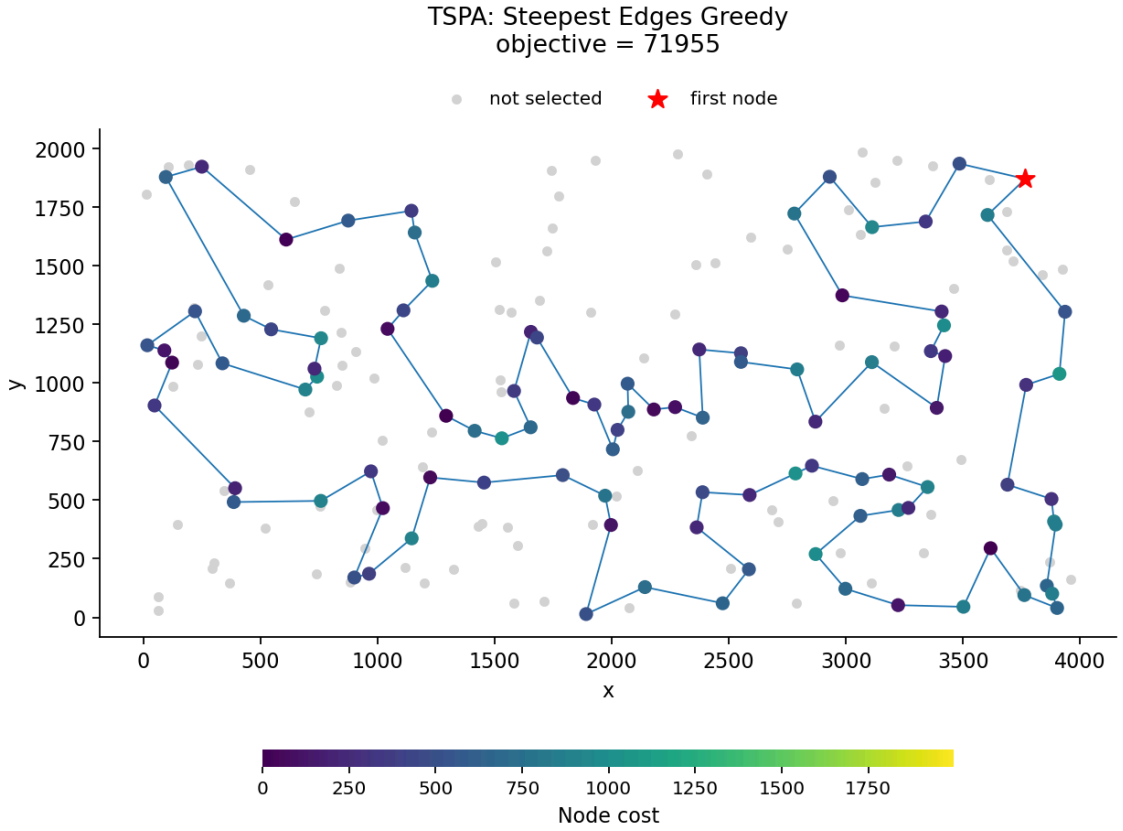


Solution (tour order):

```
127  69 171   2 189 187 161  30 141  87
 45 140  53  41  73 135 114 181 192 158
 58  84 134  72 176 150 106 113  89  42
151 182  57 107  88  78 193 153 170 146
159 130  93   3  50  47 147 169   1 178
 29 163 136 186 164  85 174   8  54   9
 33 131  36 177 179  52 144  67  16  51
156  35 185  28 180  90  83 123 100 132
 76   4  71 157  91  38  66  80  48  31
 60 108 152  49  63 149 116 168  95  97
```

Steepest Edges Greedy

Objective: **71955**.

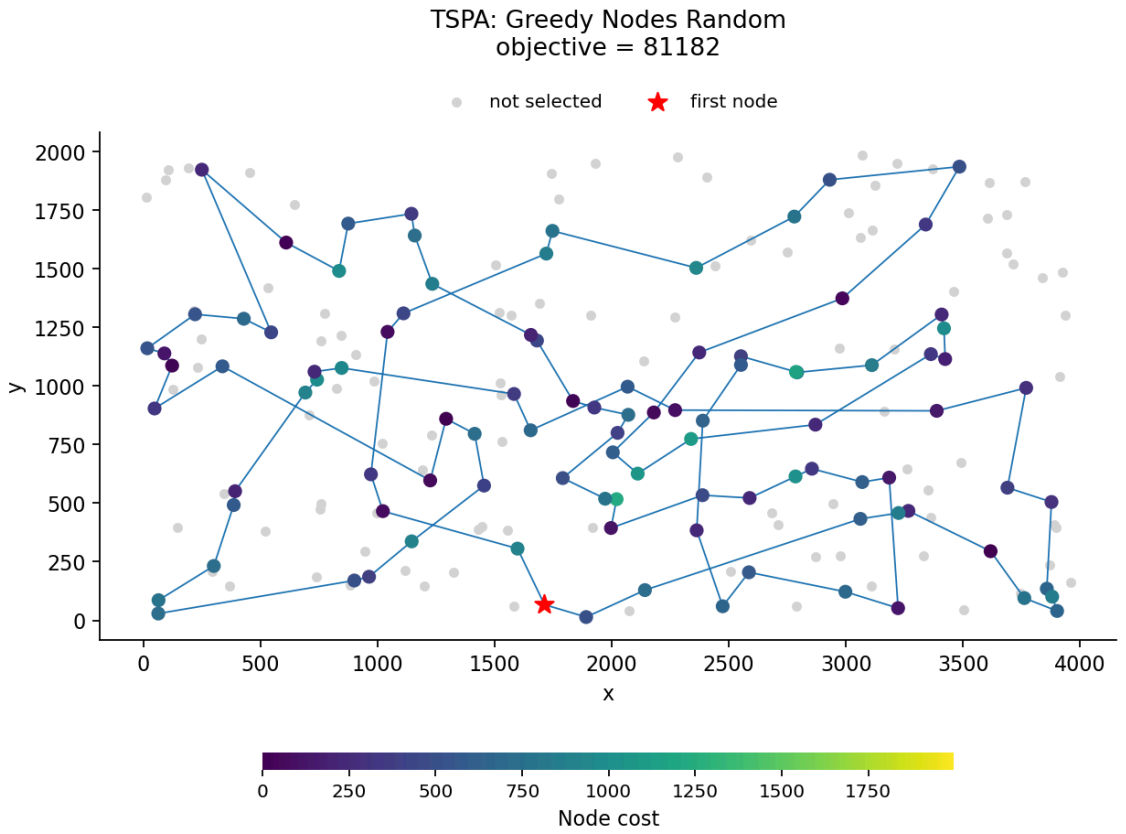


Solution (tour order):

```
 55 101 147  37  47 130 159 146 170 153
193  78  88 107  64  57 182 138 151  42
 89 113 106 150 176  72 134  84  58 158
192 181 114 135  73  45  87 141 189   2
171 187  30 161 127  97 168  63 149 116
 60  49 160  83 123 100 152 108  31  48
 80  91 157  71   4  76 132 180  28 102
185  35  51  16  67 144 140  53 179  52
177  36 131  33   9  54   8 174  85  93
 50   3 169   1 164 186 136 184 178  29
```

Greedy Nodes Random

Objective: **81182**.

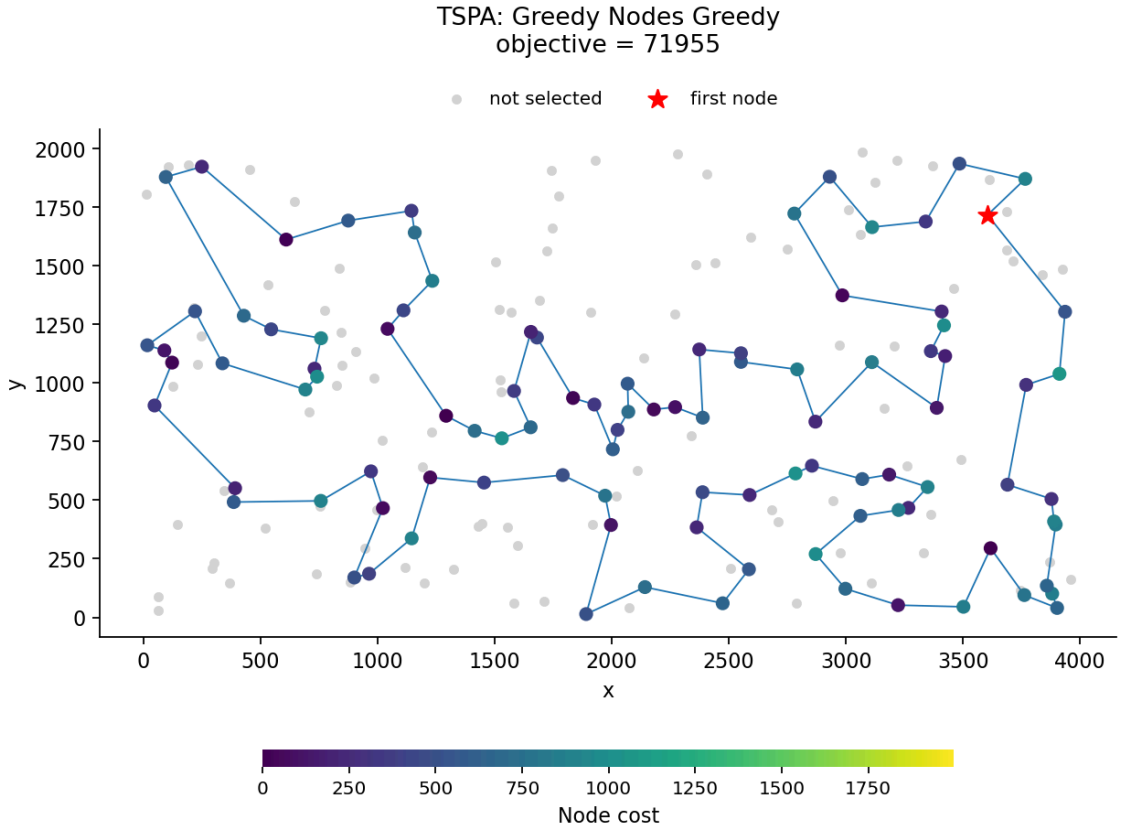


Solution (tour order):

```
  25  98 187  30 132  76 139 112  62 186
136  29 178 164  33 177 140  41  99 174
  3  50 169   1  85   8 194   9  54 131
 58 192 158 182  57 106 150 176  72 134
 84 135  61  73  45  53 179 144  67  16
 51   4  71 157  91  38  80  48 152 108
 60 116 149  63 168  49 141 180  28  87
189   2 171 104 129  69 127  97 160  83
123  90  35 185  52  36  93  47 130 159
153 193  78  88 107  89  42 151 181 114
```

Greedy Nodes Greedy

Objective: **71955**.

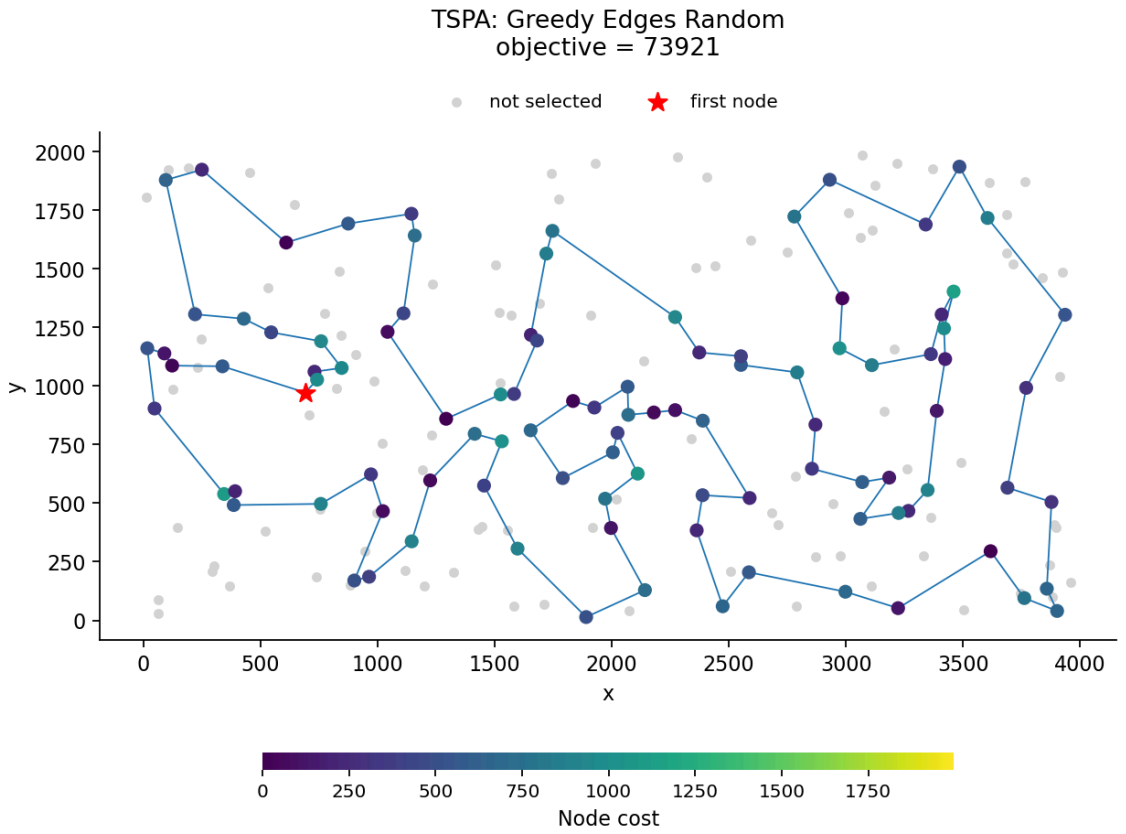


Solution (tour order):

```
101  55  29 178 184 136 186 164   1 169
  3  50  93  85 174   8  54   9  33 131
 36 177  52 179  53 140 144  67  16  51
 35 185 102  28 180 132  76   4  71 157
 91  80  48  31 108 152 100 123  83 160
 49  60 116 149  63 168  97 127 161  30
187 171   2 189 141  87  45  73 135 114
181 192 158  58  84 134  72 176 150 106
113  89  42 151 138 182  57  64 107  88
 78 193 153 170 146 159 130  47  37 147
```

Greedy Edges Random

Objective: **73921**.

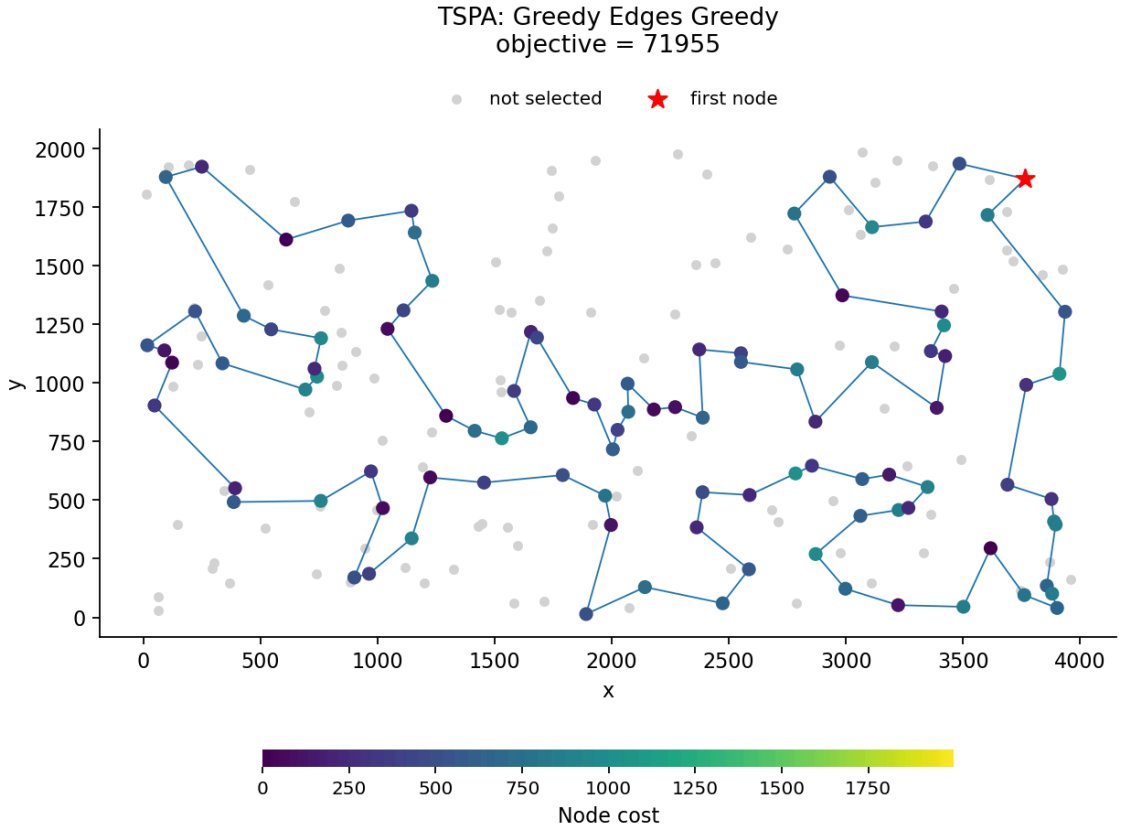


Solution (tour order):

```
160  49  63 149 116 168  95  97 127 161
 30 187 171   2 189 141  28 102  87  98
114 181 135  73  41  53 140  45 185  67
144  52 179 177  36 131 134  84  58 192
158 182  57 107  88  78 153 159 130  47
147 101  29 178 136 186 164 142  85   3
  1  59 169  50  93 113  89  42 151 106
150 176 174   8  54   9  33 143 112 139
 51  16  35 156 180 132  76  71 157  91
 80  48  31  60 108 152 100  90 123  83
```

Greedy Edges Greedy

Objective: **71955**.



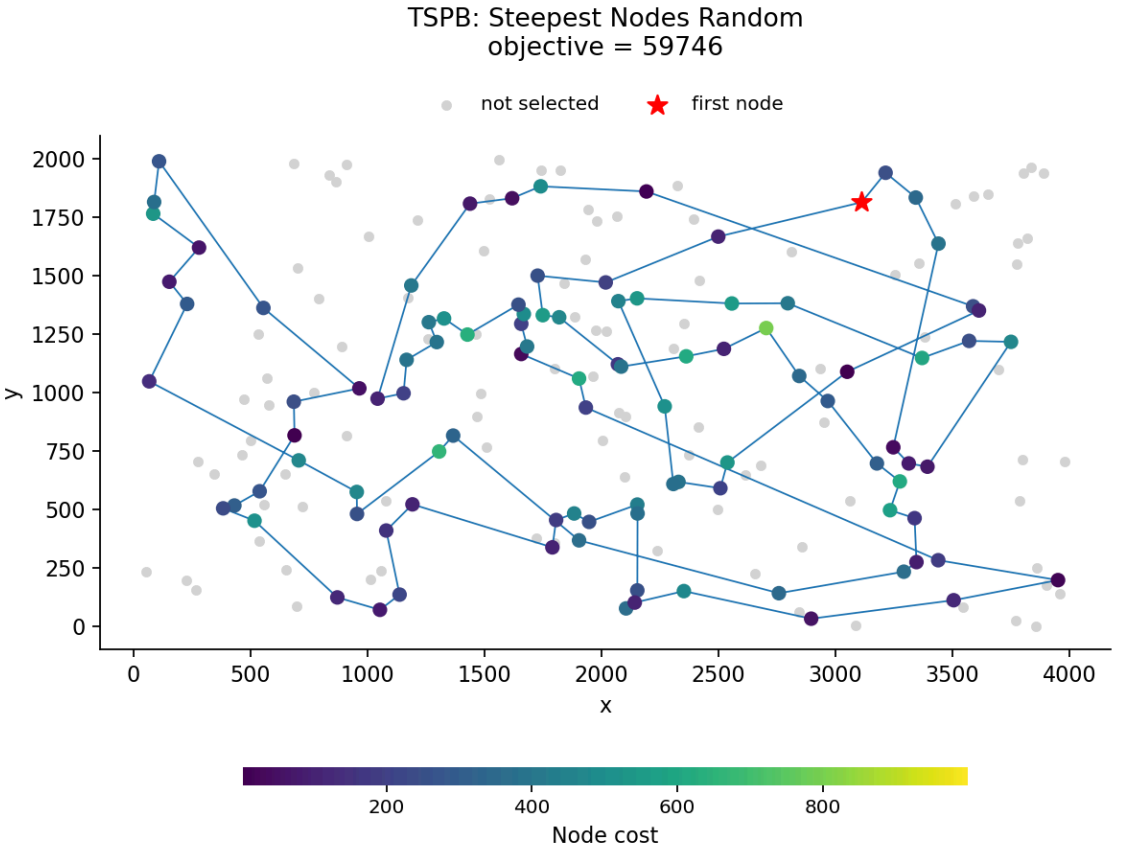
Solution (tour order):

```
 55 101 147  37  47 130 159 146 170 153
193  78  88 107  64  57 182 138 151  42
 89 113 106 150 176  72 134  84  58 158
192 181 114 135  73  45  87 141 189   2
171 187  30 161 127  97 168  63 149 116
 60  49 160  83 123 100 152 108  31  48
 80  91 157  71   4  76 132 180  28 102
185  35  51  16  67 144 140  53 179  52
177  36 131  33   9  54   8 174  85  93
 50   3 169   1 164 186 136 184 178  29
```

TSPB

Steepest Nodes Random

Objective: **59746**.

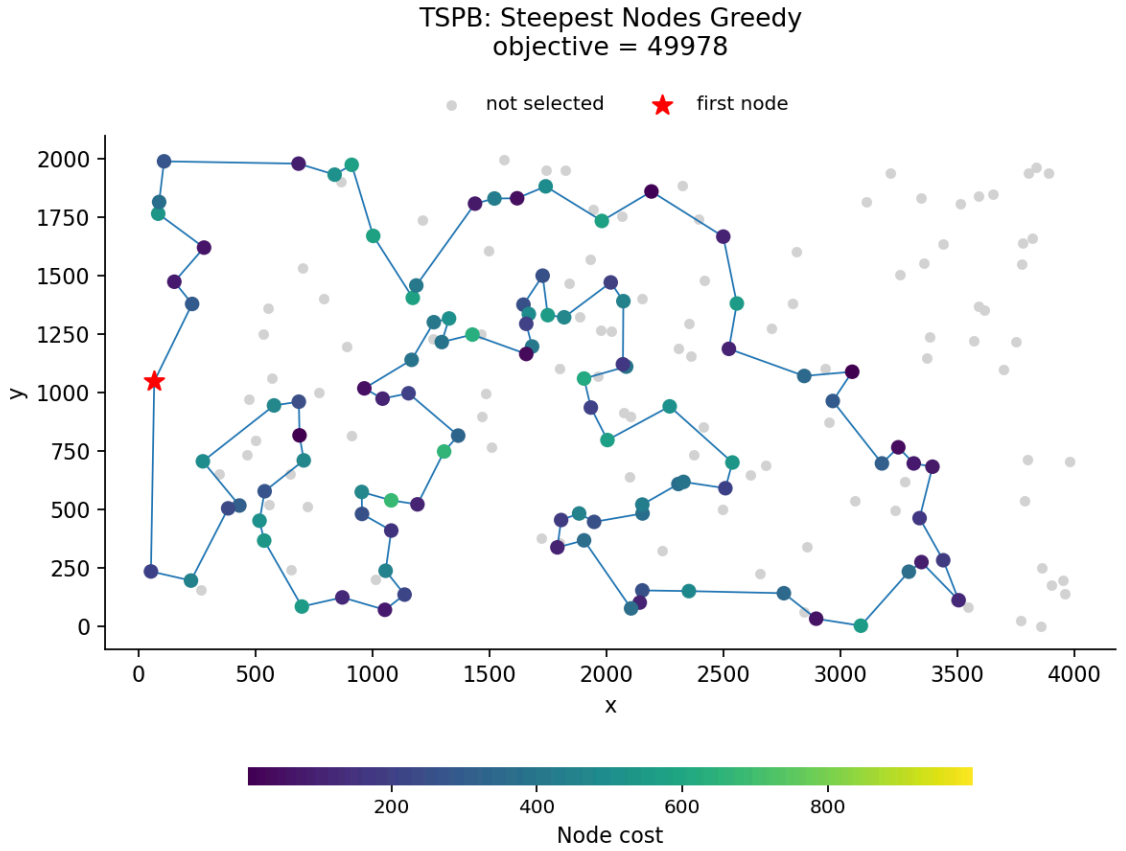


Solution (tour order):

```
194  57  72 120   8 117 184   3 135  76
144 162 192  80  52 182 153  20  74  99
 51  64  33  71 155 115  65 145  28 149
 13 146  37 158 154 125 170   6  29 132
197 175 134  45  60  40  85  59  73 167
152 181 179  82  87  88 171   2 110  14
151 122  69 160 172 136 133 189 183  30
 21  38  47 140 130 121  96  48  89 196
114  54  66  67  24  34  27  46  86 185
104  78  10 148  90 129 127 187  70 190
```

Steepest Nodes Greedy

Objective: **49978**.

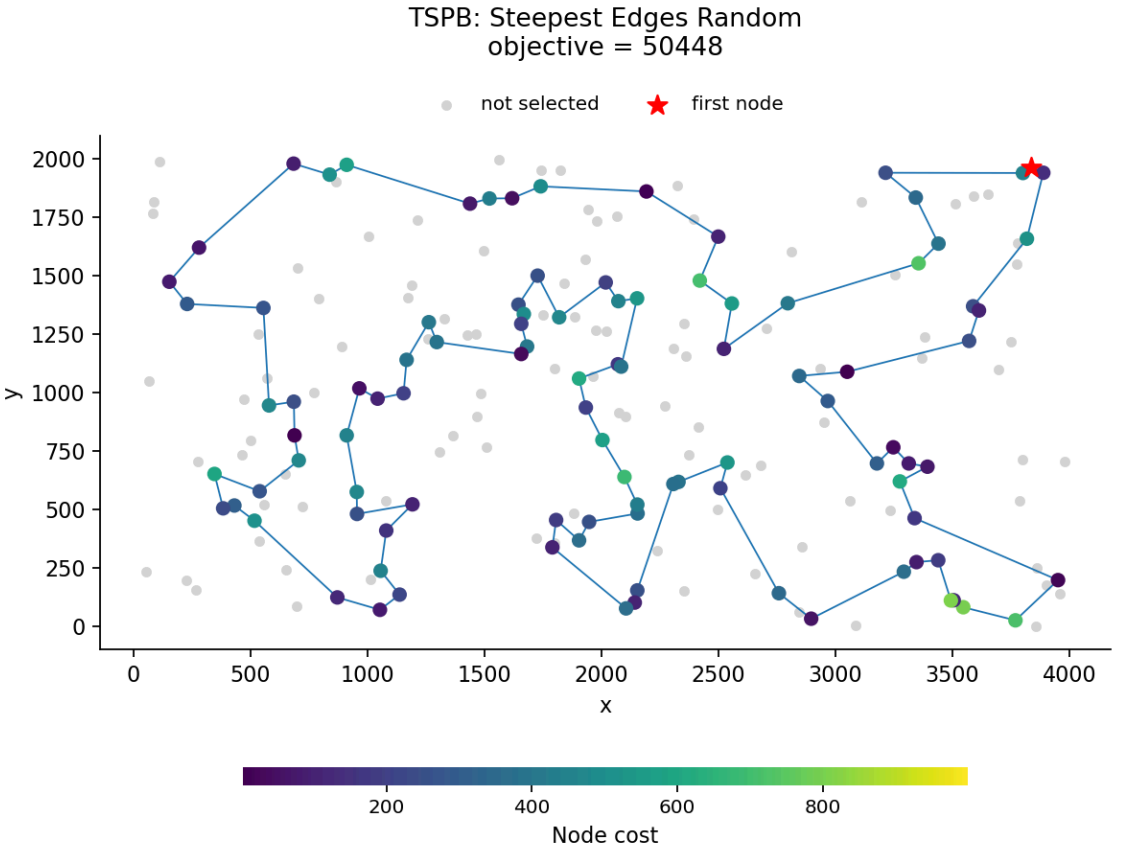


Solution (tour order):

```
 65 145  28 149  13 146  37 116 147  42
 50 191 130 121 198  96  48 156  89  57
185  76 162  54 192  80 127 129  90 153
 14   2  20  74  17 171  99  88 179  87
 82  51  85  59  73 167 181 152  34  24
 67  66  27  98 151 122   3 184  46  72
117   8 120 133 136 172 160  69 189  21
183  30  38 154 140  47  64  33  40 141
155  71  60 157  45 134 175  62  36 197
  6 115 170 125 177 178  29 132 174   1
```

Steepest Edges Random

Objective: **50448**.

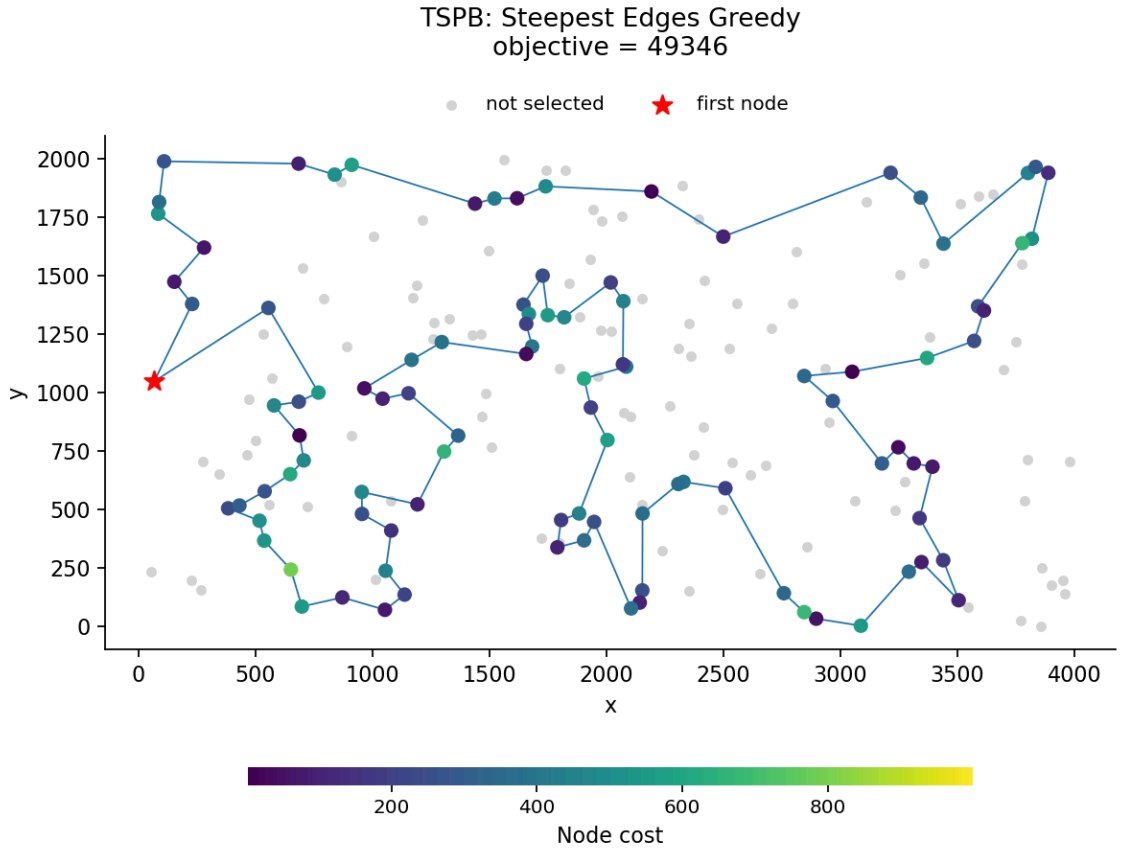


Solution (tour order):

```
 94 105  61 196 114  10  54 162 192  80
127 129  90  52 153 110  93   9   2 188
 14  20  74 171  99  67  66  24  34 179
 87  82  85  59  51 167 181 152  25  98
151 122 184   3  86  46  72 117 120 133
136 172 160  69  21  30  38  47 140 154
 97 155  71  40  60 157  45 134 175 197
 29 132 103   6 115 170 125 177 158 145
 28 149 116 147  42 121 198  96  48  89
 57  79 185  76 104  18 187  70 190  81
```

Steepest Edges Greedy

Objective: **49346**.

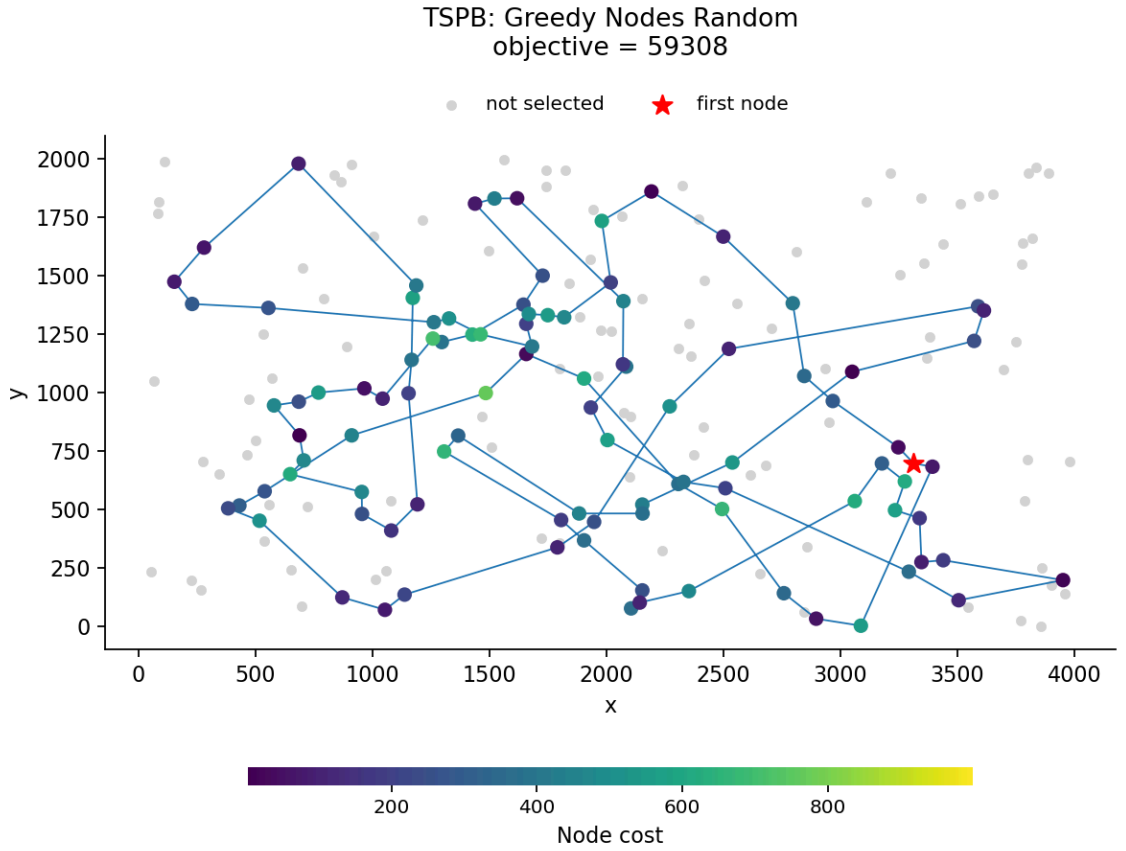


Solution (tour order):

```
 65 145  28 149  13 146  37 116 147  42
121 198  96  48  89  57 190  70 187  81
 94 105  61 139 196 114  10  78  54 162
192  80 127 129  90 153  14   2  20  74
 17 171  39  99  67  24  34 181 179  87
 82 167  51  85  59  73  98 151 122   3
184  46  72 117   8 120 133 136 172 160
 69  21  38 154 140  47  64  33  40 155
 71  60 157  45 134 175  62  91  36 197
132  29   6 180 115 170 177 125  26 158
```

Greedy Nodes Random

Objective: **59308**.

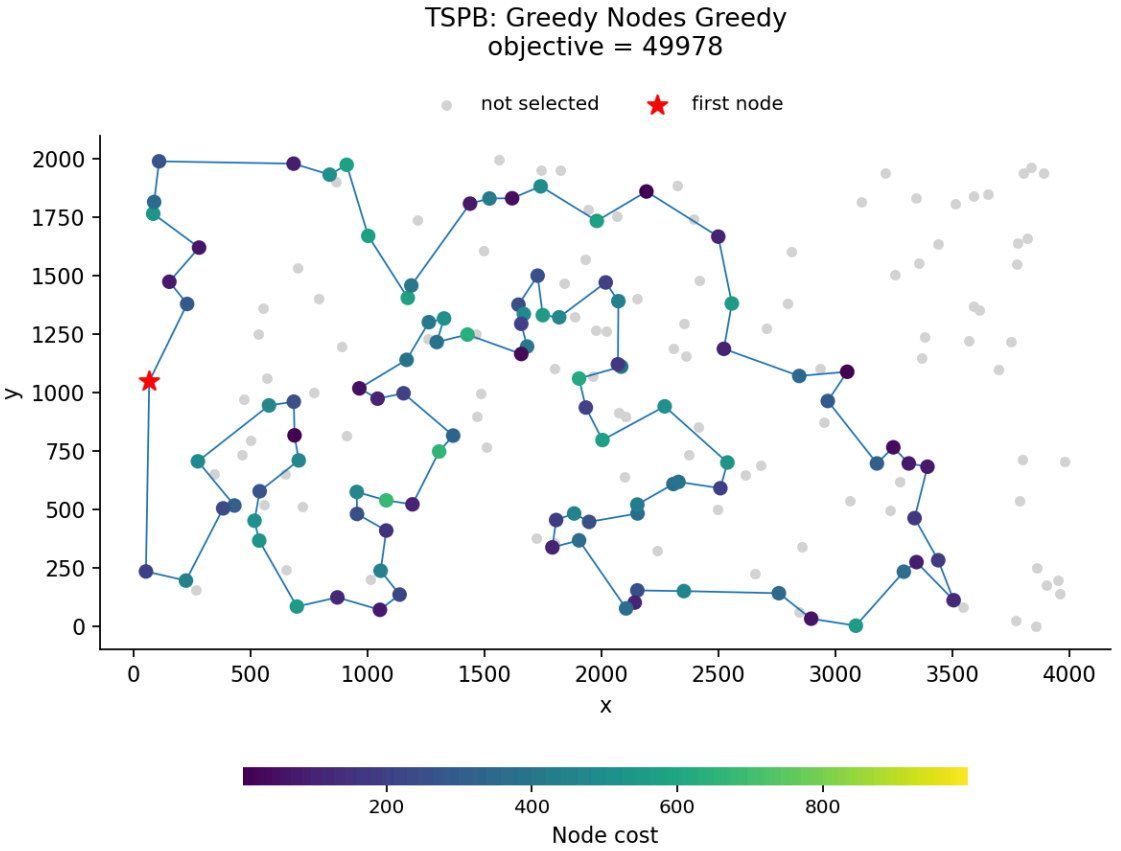


Solution (tour order):

```
129  90  17 171  99  31  34 122  69 124
 97   6  29 132 197 175 134  45  85 167
 27  76 196 114  10  54  66 152 181  73
 64  33  59  51 179  82  87  88 150  80
 52 182 153  20  14 110   2  74  67  24
 98 151   3 184  46  96 198 121 120 133
189  21 176 140 154  26 125 177 170 115
180 155  71  60  40  47  38 191 130 116
149  28 145 158  30 183 168 160 172 136
  8 117  72 156  89  57 104 162 192 127
```

Greedy Nodes Greedy

Objective: **49978**.

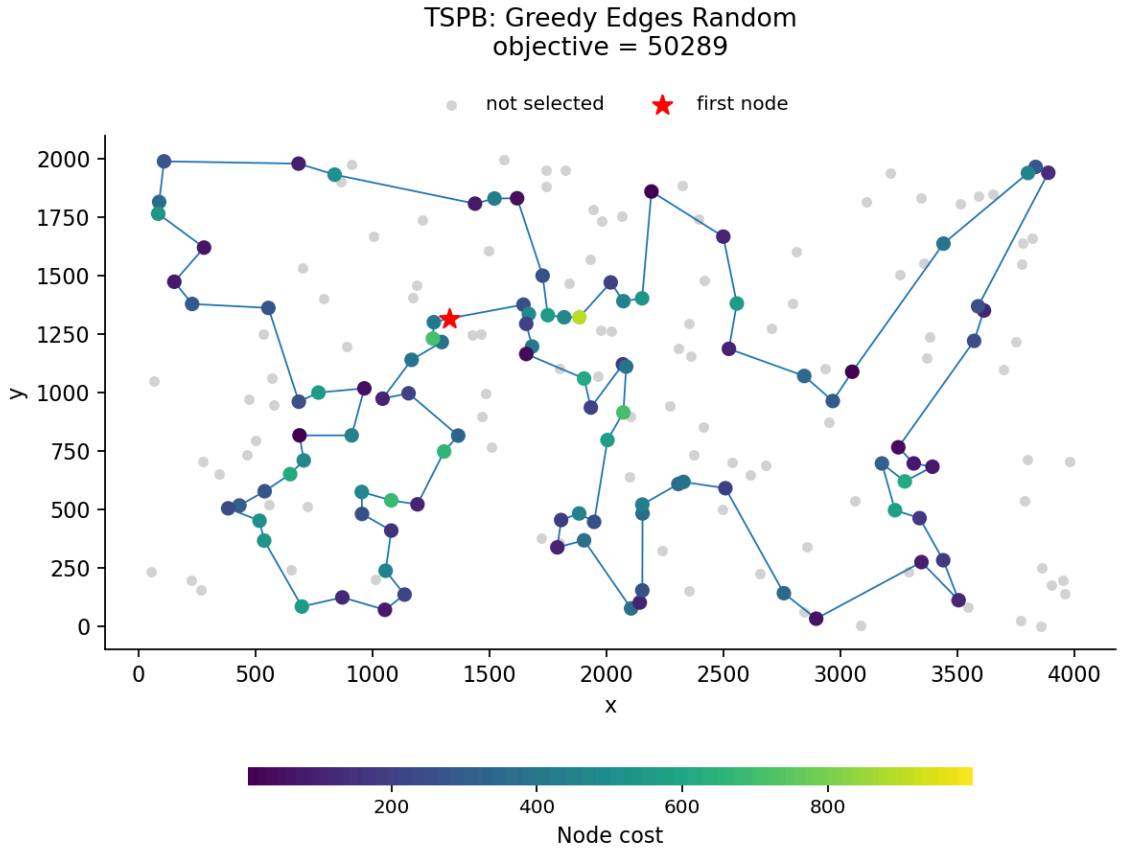


Solution (tour order):

```
  65 145  28 149  13 146  37 116 147  42
  50 191 130 121 198  96  48 156  89  57
 185  76 162  54 192  80 127 129  90 153
  14   2  20  74  17 171  99  88 179  87
  82  51  85  59  73 167 181 152  34  24
  67  66  27  98 151 122   3 184  46  72
 117   8 120 133 136 172 160  69 189  21
 183  30  38 154 140  47  64  33  40 141
 155  71  60 157  45 134 175  62  36 197
   6 115 170 125 177 178  29 132 174   1
```

Greedy Edges Random

Objective: **50289**.

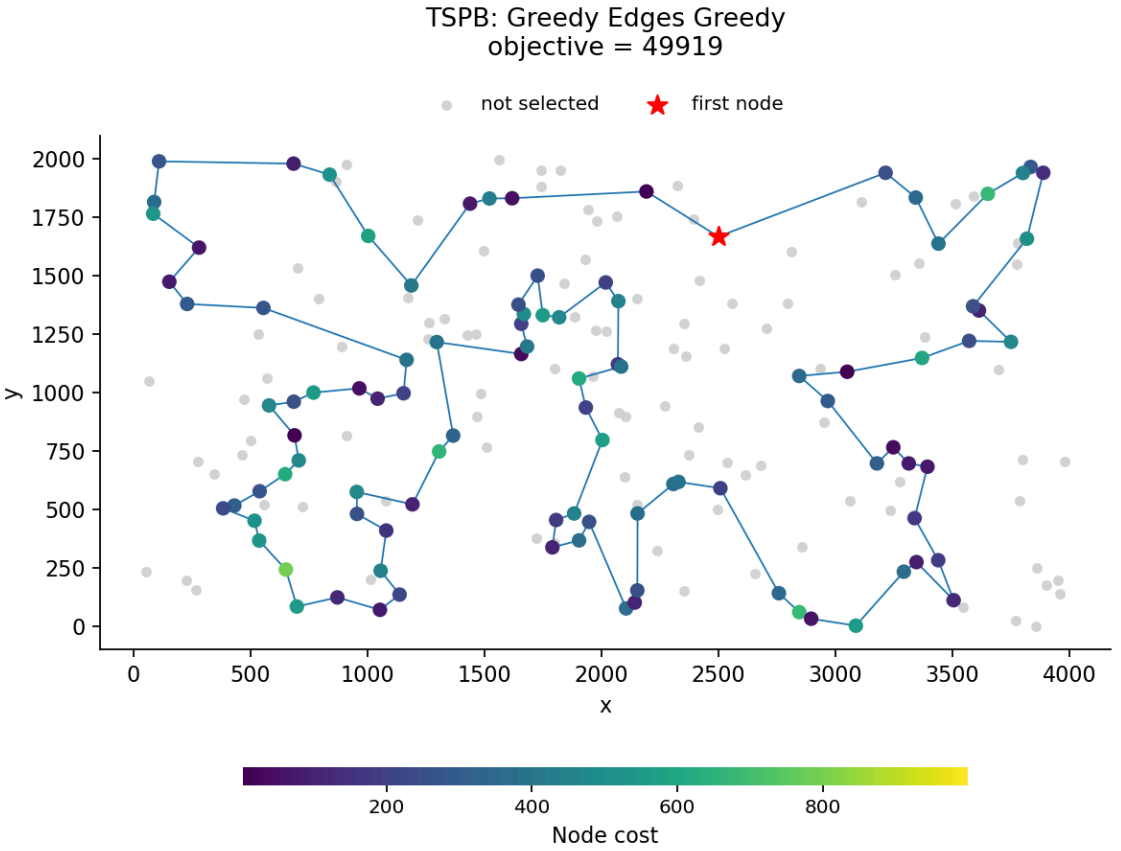


Solution (tour order):

```
183 133 136 172 160  69 122 151 184   3
111  98 167  73  59  85  51  82  87 179
181 152  34  24  67  99 171  20   2  14
153 182  80  52  90 129 127  10 114 196
105  94  81 187  54 192 162  76 185  57
 89  86  46  72  95 117   8 120  96 198
121 147 116  37 146  13 149  28 145 158
125  26 154  97 170 115 180   6  29 132
197  36  62 175 134  45 157  60  71 155
141  40  33  64  47 140  38  21 176  30
```

Greedy Edges Greedy

Objective: **49919**.



Solution (tour order):

```
 57  89  96 198 121 130  50 147 116  37
146  13 149  28 145 158  38  47 140 154
 26 125 177 170 115 180   6  29 132 197
 36  91  62 175 134  45 157  60  71 155
 40  33  64  21  69 160 172 136 133 120
  8 117  72  46 184   3 122 151  98  73
 59  85  51 167  82  87 179 181  34  24
 67  99  39 171  17  74  20   2  14 153
 90 129 127  80 192 162  54  78  10 148
114 196  61 105  94  81 169 187  70 190
```

Solution checking

All 16 best solutions were checked for exactly 100 distinct indices in the range 0-199.

Source code

Repository: PiotrFoltyniewicz/Evolutionary-Computation.

- Local search (steepest, greedy, moves, starting solutions)
- Experiment runner

- Experiment statistics and timing
- Visualization script
- Exported solutions
- Lab 3 summary

Conclusions

Local search improved the greedy starting solutions: on TSPA the best average is 72986.99 (Greedy Edges Greedy, previously 73353.51) and the best objective 71955 (previously 72011); on TSPB the best average is 52633.83 (Steepest Edges Random) and the best objective 49346 (previously 50485). The gain in the average over the greedy starting solutions is about 0.5% on TSPA and 2.6-4.7% on TSPB.

Edge exchange is clearly better than node exchange, especially from random starts (TSPA 76.1-76.2k vs 88.1-89.7k, TSPB 52.6-53.0k vs 63.9-65.9k). From greedy starts the difference is negligible on TSPA and about 1.1k on TSPB. Steepest and greedy search give similar quality from greedy starts (differences below 0.2%); from random starts greedy search is better with node exchange (by 1.8-3%) and slightly worse with edge exchange on TSPB (by 0.7%). Greedy search with edge exchange from random starts needs about twice as much local search time as steepest (about 20 ms vs 9.9 ms), probably because the whole move list is shuffled in every iteration.

Greedy starts are faster (1.6-3.0 ms in total, of which the construction takes about 1.3 ms, vs 9.9-20.7 ms from random starts) and better on TSPA, but on TSPB random starts with edge exchange have a better average and a smaller spread (maximum 55.4-55.9k vs 60.3k), although the best solutions come from greedy starts.